

Day 1 Notes — Python Practice Reset

Dataset

Titanic dataset (`train\_and\_test2.csv`) — used columns: Age, Fare, Sex, sibsp, Pclass, Embarked, Survived (target).

What I Did

1. Missing Values

- Checked missing values using `df.isnull().sum()`
- Found 2 missing values in `Embarked`
- Filled using ****mode**** (most frequent value) — because `Embarked` is categorical, mean/median don't apply to categories.

```
```python
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```
```

2. Mean, Median, Std Dev (on Age)

```
```python
age_mean = np.mean(df['Age']) # 29.5
age_median = np.median(df['Age']) # 28.0
age_std = np.std(df['Age']) # 12.9
```
```

****Interpretation:****

- Mean $\approx$ Median $\rightarrow$ data is roughly balanced, not heavily skewed
- Std Dev $\sim 13 \rightarrow$ ages are spread out fairly widely around the mean

3. Outliers in Fare (IQR Method)

```
```python
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
```

$IQR = Q3 - Q1$

$lower\_bound = Q1 - 1.5 * IQR$

$upper\_bound = Q3 + 1.5 * IQR$

`outliers = df[(df['Fare'] < lower_bound) | (df['Fare'] > upper_bound)]`

...

- Found 171 outliers, upper bound = ₹66.34

- These aren't errors — they represent 1st class/rich passengers, a genuinely different group

- Applied **\*\*log transformation\*\*** to compress the effect of extreme values:

```
```python
```

```
df['Fare_log'] = np.log1p(df['Fare'])
```

```
```
```

### ### 4. Feature Engineering — FamilySize

```
```python
```

```
df['FamilySize'] = df['sibsp'] + 1
```

```
```
```

### ### 5. Groupby — Age vs Survival

```
```python
```

```
df.groupby('Survived')['Age'].mean()
```

```
```
```

- Survived: avg age 28.3 | Did not survive: avg age 29.9

- Small difference — Age alone isn't a strong predictor of survival in this dataset; other features (Sex, Pclass) likely matter more.

## ## Concepts Learned

### ### Mean / Median / Mode / Std Dev — why they matter

- **\*\*Mean\*\***: average, but sensitive to extreme values (outliers pull it)
- **\*\*Median\*\***: middle value, resistant to outliers — good for skewed data
- **\*\*Mode\*\***: most frequent value — used for categorical data
- **\*\*Std Dev\*\***: how spread out the data is around the mean

**\*\*Used for in ML:\*\***

1. Deciding if features need scaling (e.g. Age vs Fare have very different ranges)
2. Detecting outliers before model training
3. Deciding missing-value fill strategy (mean/median for numeric, mode for categorical)
4. Judging how predictable/noisy the data is

### ### OOP Basics

- **\*\*Class\*\*** = a template/blueprint that bundles data + functions together
- **\*\*`\_\_init\_\_`\*\*** = runs automatically when a new object is created
- **\*\*`self`\*\*** = refers to "this specific object" currently being worked on
- **\*\*Argument\*\*** = extra input passed into a function/class to customize its behavior

```
```python
```

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def greet(self):
```

```
        print("Hello, my name is", self.name)
```

****Why use a class instead of plain functions?****

- Avoids repeatedly passing the same data (like a dataframe) into every function
- Keeps related data + operations bundled and organized
- Standard practice in production code — reusable, scalable